SOFTWARE REQUIREMENTS SPECIFICATION

A comprehensive structure for company projects and stakeholder submission

Practical SRS template and preparation guide

Field	Details
Project name	_____
Company / client	_____
Prepared by	_____
Document version	0.1 — Draft
Submission date	_____

Use this guide as a comprehensive starting point. Tailor it to the project's scope, risk, regulatory context, and the stakeholder's document standards. Not every section is mandatory for every project.

How to use this guide

An SRS defines what a software system is required to do, the quality and constraints it must satisfy, and how the parties will verify that the requirements have been met. It should reduce ambiguity between the stakeholder, product owner, developers, testers, security reviewers, and operations teams.

Important distinction: requirements describe expected behavior and constraints. A separate Software Design Description (SDD) can hold detailed implementation decisions, component design, and low-level architecture.

Contents

- Front matter and document control
- 1. Introduction
- 2. Overall system description
- 3. Functional requirements
- 4. Non-functional requirements
- 5. Data requirements and data management
- 6. External interfaces and integrations
- 7. System architecture and technical constraints
- 8. AI and analytics requirements (if applicable)
- 9. Testing and verification requirements
- 10. Deployment, operations, and support
- 11. Risks, issues, assumptions, and dependencies
- 12. Project delivery and governance
- 13. Requirements traceability
- 14. Appendices and supporting artifacts
- Professional writing practices
- Final submission checklist
- Suggested document sizing and references

> Recommended principle: write requirements that are necessary, clear, unambiguous, feasible, uniquely identified, traceable, and verifiable. Agree all numerical targets with the stakeholder rather than inventing them.

Front matter: document information and approvals

Cover page

- Project name and project ID; document title: Software Requirements Specification (SRS).
- Company/client, department or business unit, and authorized logo if applicable.
- Prepared by, submitted to, version, status (Draft / Under Review / Approved), and submission date.
- Confidentiality classification where applicable.

Document control

Field	Example / value to record
Document ID	SRS-PROJ-001
Version and status	1.0; Draft / For Review / Approved
Document owner	Business Analyst / Product Owner
Reviewers	Stakeholder, Technical Lead, QA Lead, Security as relevant
Approval authority	Authorized client representative
Last updated	DD-MMM-YYYY

Revision history

Version	Date	Author	Change description
0.1	Date	Analyst	Initial draft
0.2	Date	Project team	Added requirements and stakeholder feedback
1.0	Date	Document owner	Approved baseline

Approval and sign-off

Capture each approver's name, role, decision, date, and approval record. Include an approval statement confirming that the document represents the agreed requirements baseline.

Table of contents and lists

Generate the table of contents automatically from Word heading styles. For a long SRS, include a list of figures, tables, and abbreviations.

1. Introduction

Explain why the project exists, what the SRS covers, and how the document will be used.

1.1 Purpose of the document

Identify the audience, system and release covered, and how the SRS supports development, testing, estimation, stakeholder review, and acceptance. State whether it is a draft or proposed baseline.

1.2 Project background

Describe the current business situation, existing process or system, limitations, and the reason a new system or enhancement is needed.

1.3 Problem statement

State the business problem without prematurely prescribing a technical solution. Example: “The current reporting process requires manual consolidation from multiple sources, increasing turnaround time and error risk.”

1.4 Project objectives

Use measurable objectives where possible, such as reducing processing time, enabling a workflow for authorized users, improving traceability, or meeting a specified service target. Confirm targets with the stakeholder.

1.5 Project scope

Separate in-scope features and deliverables, out-of-scope exclusions, and potential future enhancements. Be explicit about what is not committed for the current release.

1.6 Stakeholders and user roles

Identify business sponsor, product owner, end users, project manager, technical lead, QA, security/compliance, operations/support, and external parties. Record their interests, decision rights, and approval responsibilities.

1.7 Definitions and acronyms

Define terms that could be interpreted differently, such as SRS, API, SLA, RBAC, PII, RTO, and RPO.

1.8 References and related documents

List accessible business requirements, PRD, statement of work, existing-system documentation, API specifications, organizational policies, and applicable standards.

2. Overall system description

2.1 Product perspective

Explain whether the system is new, a replacement, an extension, or an integration. Identify system boundaries, external actors, existing systems, and major data exchanges.

2.2 Product overview and major capabilities

List modules and their purpose—for example authentication, user management, core workflows, search/reporting, notifications, administration, and audit.

2.3 User classes and characteristics

For each role, define tasks, access level, technical expertise, usage frequency, accessibility needs, and information restrictions.

2.4 Operating environment

Specify supported browsers/devices, operating systems, server/cloud environment, databases, network conditions, third-party services, and dev/test/staging/production environments as relevant.

2.5 Assumptions and dependencies

Assumptions are believed to be true but need validation; dependencies are external items the project relies on. Record owner and impact if a critical assumption is false.

2.6 Constraints

Record budget, delivery dates, mandatory platforms, existing infrastructure, licensing, data residency, organizational security policy, and applicable regulatory obligations.

2.7 Business processes and user workflows

Use flowcharts, activity diagrams, or swimlanes to show who starts an action, decisions and approvals, exceptions, and end states.

2.8 System states and modes

Where relevant, define states such as draft/submitted/approved/rejected, pending/processing/completed/failed, or online/degraded/maintenance, including valid transitions and permissions.

3. Functional requirements

The core of the SRS: specify what the system must do, not merely the screens or technologies it uses.

3.1 Feature and module inventory

Module ID	Module	Purpose	Priority
MOD-01	Authentication	Secure user access	Must
MOD-02	User management	Manage users and roles	Must
MOD-03	Core workflow	Execute the main business process	Must
MOD-04	Reporting	Generate business reports	Should
MOD-05	Notifications	Notify users of events	Confirm

3.2 Detailed functional requirement template

- Unique ID and title.
- Requirement statement: one clear, specific, testable behavior.
- Actor, trigger, and preconditions.

- Main flow and alternate/exception flows.
- Business rules, inputs, outputs, and validation.
- Priority, dependencies, and related requirements.
- Acceptance criteria and verification method.

Example: FR-AUTH-001 — User login

Requirement: The system shall authenticate a registered user using the organization's approved authentication mechanism before granting access to protected application features.

Preconditions: The user has an account and the authentication service is configured.

Main flow: (1) User opens login. (2) User provides required credentials or completes the configured identity-provider flow. (3) System validates the authentication response. (4) System establishes an authenticated session and applies assigned permissions.

Exceptions: Invalid credentials deny access with a generic error; unavailable authentication service returns an appropriate error without granting access; disabled accounts are denied according to policy.

Acceptance criteria: Valid authentication grants access only to authorized features; invalid authentication does not grant access; protected endpoints reject unauthenticated requests; errors do not reveal sensitive authentication details.

Verification: Positive, negative, and authorization tests.

3.3 Use cases and user stories

For complex workflows, specify actor, goal, preconditions, normal flow, alternate flow, and outcome. User-story format: “As a [role], I want to [action], so that [benefit].” Add acceptance criteria and business rules; a user story alone may be insufficient for complex or regulated behavior.

3.4 Business rules

Record rules for approvals, limits, calculations, deadlines, required fields, duplicate handling, and state transitions. Assign stable IDs such as BR-001.

3.5 Validation and exception handling

Specify behavior for malformed input, duplicate submissions, invalid state transitions, unauthorized actions, timeouts, unavailable dependencies, partial failures, concurrent updates, rate limits, invalid uploads, retries, and recovery.

3.6 Roles and permissions

Action	Standard user	Manager	Administrator
View own records	Yes	Yes	Policy-defined
Submit a request	Yes	If authorized	If authorized
Approve requests	No	Within assigned scope	Policy-defined
Manage user accounts	No	No	Yes

Use the stakeholder's real access-control policy. Specify where enforcement occurs—backend/API, identity provider, data layer, and UI as appropriate.

3.7 Search, filtering, and reporting

Define searchable fields, filters, sorting, pagination, export formats, calculations, time-zone handling, data freshness, access restrictions, and empty-state behavior.

3.8 Notifications and integrations

Specify triggers, recipients, channels, content, retries, duplicate prevention, delivery status, and integration request/response, authentication, timeout, and recovery behavior.

3.9 Acceptance criteria

Use measurable conditions. Given a valid user with permission, when the user submits a valid request, then the system saves it and returns a unique reference ID. Also specify expected behavior for invalid input, insufficient permissions, and unavailable dependencies.

4. Non-functional requirements

Define measurable quality attributes and operating limits. Avoid vague statements such as “fast,” “secure,” or “highly scalable.”

4.1 Performance

Specify API/page latency, concurrent users, requests per second, batch duration, database queries, upload/download time, report generation, and peak-load behavior. Example: under an agreed reference workload of 500 concurrent users, 95% of standard read API requests complete within 2 seconds, excluding explicitly identified external-service waiting time. Confirm workload, environment, measurement, and exclusions.

4.2 Scalability

Define expected growth in users, organizations, data volume, transactions, and traffic; scaling approach where constrained; maximum workload; and acceptable degradation.

4.3 Availability and reliability

Define service availability, permitted downtime, maintenance windows, failure detection, recovery, retry policy, consistency expectations, and partial-failure handling. A 99.9% monthly target corresponds to about 43 minutes of downtime in a 30-day month if the entire month is counted; define exclusions and measurement.

4.4 Security

Cover authentication, authorization, least privilege, session management, encryption in transit/at rest, secrets, input validation, common web/API attacks, vulnerability management, audit trails, safe errors, backup protection, and incident response.

4.5 Privacy and data protection

Define data categories, purpose and permitted use, minimization, consent where applicable, retention/deletion, access/correction/export where required, masking, residency, third-party sharing, and applicable laws/contracts.

4.6 Usability and UX

Specify navigation consistency, error messages, learnability, validation feedback, responsive layouts, accessibility, localization, keyboard navigation, screen-reader support, onboarding, and help. Set usability-study targets where useful.

4.7 Accessibility

If applicable, identify the target standard and conformance level (for example WCAG 2.2 AA) and cover keyboard use, focus order, contrast, semantic structure, labels, alternative text, and error identification.

4.8 Maintainability and extensibility

Specify modularity expectations, code review, documentation, automated testing, configuration management, dependency updates, feature extensibility, backward compatibility, and internal supportability.

4.9 Compatibility and portability

List supported browser/OS versions, device sizes, API clients and versions, database versions, deployment environments, and import/export formats.

4.10 Observability and auditability

Define logs, error rates, latency metrics, health checks, distributed traces where relevant, business-event audit records, alert destinations, correlation IDs, dashboards, retention, and access controls. Distinguish diagnostic logs from audit records.

4.11 Backup and disaster recovery

Specify backup frequency/retention, Recovery Point Objective (RPO: maximum acceptable data loss measured in time), Recovery Time Objective (RTO: target time to restore service), recovery environment, restore testing, responsibilities, and integrity checks.

4.12 Resource utilization and cost

Where relevant, define CPU/memory/storage/network limits, cloud budget, third-party API quotas, cost alerts, and resource-cleanup requirements.

4.13 Environmental and operational constraints

Include deployment region, approved infrastructure, operating conditions, physical constraints, network restrictions, or organizational environment requirements where applicable.

5. Data requirements and data management

5.1 Data inventory and classification

For each data category, record source, owner, sensitivity, purpose, and lifecycle. Examples include profiles, transactions, business records, uploads, audit events, and analytical outputs.

5.2 Data model and relationships

Include an ER diagram or equivalent showing entities, attributes, keys, relationships/cardinalities, required/optional fields, uniqueness, and referential integrity. Detailed physical schema can be separate.

5.3 Data dictionary

Define important fields consistently. Example: user_id (UUID, required, unique); created_at (timestamp, required, UTC); status (enum, allowed values defined); amount (decimal, precision and range defined).

5.4 Validation and integrity

Specify mandatory fields, formats/ranges, uniqueness, duplicate detection, referential integrity, related-record consistency, concurrent updates, reconciliation, and invalid/incomplete record handling.

5.5 Migration and import

Identify source systems, mapping, transformation, historical scope, duplicate handling, reconciliation, cutover, rollback, and migration sign-off.

5.6 Retention, archival, and deletion

Define retention periods by data type, archival conditions, deletion behavior, and legal holds or audit restrictions.

5.7 Data quality and freshness

For analytics/AI, specify completeness, accuracy, consistency, timeliness, provenance, and refresh frequency.

5.8 Data ownership and access

Define data owners, who can view/change datasets, how access is approved, and how access is reviewed or revoked.

6. External interfaces and integrations

6.1 User interfaces

Document expected screens, user flows, design references, and behavior. Attach sitemap, wireframes, Figma links, screen inventory, form fields, navigation, responsive behavior, and loading/success/error/empty states. Final visual styling may live in a separate UI/UX specification.

6.2 API interfaces

- Endpoint and HTTP method; purpose; authentication.
- Request and response schemas; status/error codes; validation.
- Pagination/filtering; rate limits; timeout/retry; idempotency where needed.
- Versioning and backward compatibility.

Reference an OpenAPI specification rather than duplicating all schemas when practical.

6.3 Third-party integrations

Item	What to specify
Service	Name and business purpose
Data exchanged	Request and response data
Authentication	Approved mechanism
Availability	Dependency assumptions or contractual SLA
Failure behavior	Timeout, retry, fallback, graceful degradation
Ownership	Internal and external contact
Testing	Sandbox, mocks, integration tests

6.4 Hardware and device interfaces

Include only when the system interacts with devices such as sensors, printers, scanners, terminals, or other hardware.

6.5 Communication interfaces

Define protocols, network requirements, message formats, event-delivery guarantees, and transport-security requirements.

6.6 Integration diagram

Show the application, databases, identity provider, third-party APIs, message brokers, storage, and monitoring services as relevant.

7. System architecture and technical constraints

An SRS states required behavior and constraints; a Software Design Description (SDD) explains the implementation in detail. A high-level architecture section is useful when it clarifies the system boundary.

7.1 System context diagram

Show the system boundary, users, external systems, and major data exchanges.

7.2 High-level architecture diagram

Show frontend, backend services, database, authentication, background workers, external integrations, and logging/monitoring as applicable.

7.3 Technology and platform constraints

Specify mandatory languages, frameworks, cloud platforms, licensing restrictions, or enterprise standards only where genuinely required.

7.4 Deployment and environment model

Describe environments, deployment topology, network boundaries, and separation of development, test, staging, and production.

7.5 Design constraints and decisions

Record constraints and rationale. Keep detailed design rationale in architecture decision records if used.

7.6 Component boundaries

Clarify ownership of responsibilities and where critical requirements are enforced. Do not leave ambiguity about whether authorization is enforced by frontend, backend, identity provider, or data layer.

8. AI and analytics requirements (if applicable)

Include this section only if the project uses an LLM, machine learning, recommendation engine, forecasting, or automated decision-making.

8.1 Model and feature requirements

Specify intended and prohibited uses, input data, output, supported use cases, dependencies, behavior on insufficient input, meaningful confidence thresholds, human review, and fallback behavior.

8.2 AI quality and evaluation

Define use-case-appropriate metrics such as accuracy, precision, recall, task success, groundedness, or retrieval quality. Record datasets, conditions, thresholds, and approver.

8.3 AI safety, privacy, and security

Consider prompt injection, untrusted input, data leakage, unauthorized tool calls, sensitive-data exposure, hallucinations, output validation, retrieval permissions, and human oversight for high-impact decisions.

8.4 RAG requirements

Specify authorized sources, ingestion/update frequency, chunking/retrieval expectations, access-control enforcement, citations/source display, behavior when evidence is unavailable, and evaluation.

8.5 Model lifecycle and monitoring

Define approval, evaluation, deployment, versioning, monitoring, rollback, and retirement. State measurable expectations rather than promising that a model will always be correct.

9. Testing and verification requirements

9.1 Testing strategy

- Unit, integration, system, end-to-end, and regression testing.
- Performance/load, security, and accessibility testing where applicable.
- User Acceptance Testing (UAT).
- Disaster-recovery testing where relevant.

Detailed scripts and procedures can live in a separate test plan.

9.2 Verification methods

Method	Meaning
Test	Execute software and evaluate results
Inspection	Examine a document, configuration, interface, or artifact
Analysis	Use calculations, models, or analytical evidence
Demonstration	Show the required behavior under defined conditions

9.3 Test environment and data

Document required environments, test accounts, representative or synthetic datasets, integration sandboxes, and restrictions on production data.

9.4 Acceptance criteria

- All mandatory requirements pass their agreed verification.
- No unresolved critical security defects remain, subject to the agreed exception process.
- Performance targets pass under the specified workload.
- Data migration reconciliation is complete where relevant.
- Stakeholder UAT approval is recorded.
- Required documentation and operational handover are complete.

Define how high-severity defects, approved exceptions, and deferred issues are handled.

9.5 Requirements verification matrix

Requirement ID	Summary	Method	Expected evidence	Status
FR-AUTH-001	Authentication	Test	Authentication test results	Planned
FR-USER-002	Permissions	Test	Role-access test results	Planned
NFR-PERF-001	Latency target	Performance test	Load-test report	Planned
NFR-REC-001	Recovery objective	Test/demo	Recovery-test report	Planned

10. Deployment, operations, and support

10.1 Deployment requirements

Define environments, release process, configuration and secrets, database migrations, rollback, approvals, deployment verification, and downtime constraints.

10.2 CI/CD and release management

Where relevant, define build, automated test, security scan, approval, artifact, and deployment requirements without mandating a tool unnecessarily.

10.3 Monitoring and alerting

Define health indicators, alert thresholds, escalation, on-call responsibilities, and conditions that open an incident.

10.4 Incident management

Specify severity, response expectations, escalation, communication, recovery, and post-incident review.

10.5 Maintenance and support

Include support hours/ownership, defect triage, patching, dependency maintenance, documentation, end-user support, handover, and decommissioning where applicable.

10.6 SLOs and SLAs

Distinguish internal Service Level Objectives from contractual Service Level Agreements. Record measurement windows, exclusions, responsibilities, escalation, and remedies where applicable.

11. Risks, issues, assumptions, and dependencies

ID	Risk	Impact	Mitigation	Owner
R-001	External API availability uncertain	High	Confirm SLA; define fallback	Integration lead
R-002	Source data quality unknown	High	Profile sample data early	Data lead
R-003	Requirements may change	Medium	Use formal change control	Product owner
R-004	Performance targets unvalidated	Medium	Run baseline tests	Technical lead

Distinguish: a risk is an uncertain event; an issue has already occurred; an assumption is believed true but needs validation; a dependency is an external requirement; an open question is an unresolved decision.

For significant items, record owner, due date, impact, status, and resolution plan.

12. Project delivery and governance

12.1 Deliverables

List agreed outputs: application, source code/repository handover, API documentation, test reports, deployment configuration, user and administrator manuals, training, operational runbooks, security/compliance evidence where applicable. Record owner, due date, acceptance criteria, and recipient.

12.2 Milestones and phased delivery

Phase	Deliverables	Exit condition
Discovery	Requirements and stakeholder validation	Scope approved
MVP	Mandatory workflows	MVP acceptance criteria passed
Integration	External services and data flows	Integration tests passed
UAT	Stakeholder validation	UAT approval
Production	Release and handover	Go-live criteria met

12.3 Change management

- Assign change request ID and describe the proposed change.
- Record business justification.
- Assess cost, schedule, architecture, testing, security, and requirement impact.
- Record decision maker, approval status, and decision.
- Update affected requirement IDs, version, and linked artifacts.
- Communicate approved changes to affected teams.

Baseline approved requirements and manage changes through the project’s document/configuration management process.

12.4 Roles and responsibilities

Define ownership for requirements, technical decisions, verification, security review, deployment, and acceptance. A RACI matrix can help on larger projects.

12.5 Training and knowledge transfer

Specify audience, materials, delivery method, sessions, and completion evidence where applicable.

13. Requirements traceability

Trace business needs to requirements, design/implementation, verification evidence, and acceptance. Traceability helps reveal requirements without tests, tests without requirements, and features that do not serve an agreed business need.

Requirement ID	Business objective	Design reference	Test case	Status
FR-AUTH-001	OBJ-001	DESIGN-AUTH-01	TC-AUTH-001	Planned
FR-USER-002	OBJ-002	DESIGN-USER-02	TC-USER-005	Planned
NFR-PERF-001	OBJ-003	DESIGN-API-01	TC-PERF-001	Planned

Use stable IDs across the SRS, tickets, design documents, tests, and defects wherever practical.

14. Appendices and supporting artifacts

- Appendix A — Glossary and terminology.
- Appendix B — Use-case diagrams.
- Appendix C — Workflow diagrams.
- Appendix D — Wireframes and screen references.
- Appendix E — Data dictionary.
- Appendix F — API specifications.
- Appendix G — Architecture and context diagrams.
- Appendix H — Full requirements traceability matrix.
- Appendix I — Risk and dependency register.
- Appendix J — Open questions and decision log.
- Appendix K — Compliance mapping, where applicable.
- Appendix L — Approval record.

For large projects, keep detailed API specifications, test plans, design documents, and runbooks as separate controlled documents and reference their exact versions from the SRS.

15. Professional requirements-writing practices

15.1 Write verifiable requirements

Weak wording	Better wording
The system should be fast.	95% of requests under the defined workload shall meet the agreed latency target.
The system should be secure.	Unauthenticated requests to protected endpoints shall be denied.
The system should be user-friendly.	Users in the defined study shall complete the workflow at the agreed success rate.
The system should support many users.	The system shall support the agreed concurrent-user load while meeting stated performance targets.
The system should handle errors.	On provider timeout, the system shall show an indeterminate status and prevent duplicate submission during reconciliation.

Targets must be validated with the stakeholder. Do not invent numbers simply to make the SRS appear measurable.

15.2 Requirement ID conventions

Prefix	Meaning
OBJ	Business objective
FR	Functional requirement
NFR	Non-functional requirement
BR	Business rule
UI	User-interface requirement
API	API or integration requirement
DATA	Data requirement
SEC	Security requirement
OPS	Operational requirement
TC	Test case
RISK	Risk register item

A single NFR prefix plus a category field is also acceptable. Consistency matters more than the specific convention.

15.3 Prioritization

- Must have: essential for the agreed release.
- Should have: important, but a viable release is possible without it.
- Could have: desirable if time and resources permit.
- Won’t have this release: explicitly excluded from current scope.

15.4 Open questions and assumptions

ID	Question / decision	Owner	Due date	Status
Q-001	What is the maximum expected concurrent-user load?	Stakeholder	Date	Open
Q-002	Which identity provider is required?	IT lead	Date	Open
Q-003	What is the required data-retention period?	Compliance	Date	Open

15.5 Version control

Use a versioned repository or document-management system. Make the latest approved baseline easy to distinguish from older drafts.

16. Final stakeholder-submission checklist

A. Document quality

- ☐ Cover page, document ID, version, and status
- ☐ Revision history and document owner
- ☐ Table of contents and consistent formatting
- ☐ References, glossary, and abbreviations

B. Business and scope

- ☐ Problem statement and measurable objectives
- ☐ Clear in-scope and out-of-scope boundaries
- ☐ Stakeholders, user roles, and approval authority

- ☐ Assumptions, constraints, and dependencies

C. Requirements

- ☐ Unique IDs for all requirements
- ☐ Functional requirements for agreed features
- ☐ Business rules, alternate flows, and error handling
- ☐ Measurable non-functional requirements
- ☐ Priorities and acceptance criteria

D. Technical and data

- ☐ System context and integration diagrams
- ☐ Data model and data dictionary where relevant
- ☐ API and external interface specifications
- ☐ Security, privacy, and access-control requirements
- ☐ Deployment, monitoring, and recovery requirements

E. Verification and delivery

- ☐ Verification method for each important requirement
- ☐ Requirements traceability matrix
- ☐ Test strategy and acceptance conditions
- ☐ Risks, open questions, and change control
- ☐ Deliverables, milestones, and operational handover

F. Stakeholder approval

- ☐ Business and technical review completed
- ☐ Conflicting or ambiguous requirements resolved
- ☐ Unapproved assumptions clearly identified
- ☐ Required sign-off recorded
- ☐ Approved baseline and document version established

17. Suggested document sizing and references

Page counts are planning estimates, not requirements. Scale the document to the project's complexity and risk.

Section	Suggested depth
Cover, control, approvals	2–4 pages
Introduction and scope	2–4 pages
Overall system description	2–5 pages
Functional requirements	As needed; often the largest section
Non-functional requirements	3–8 pages
Data and interfaces	As needed
Architecture and diagrams	2–6 pages
Verification and acceptance	2–5 pages
Operations, risks, governance	2–5 pages
Traceability, appendices, sign-off	As needed

Useful references

- ISO/IEC/IEEE 29148:2018 — Requirements engineering: iso.org/standard/72089.html.

- NASA Software Engineering Handbook — Software Requirements: swehb.nasa.gov.
- NASA Systems Engineering Handbook: nasa.gov/reference/system-engineering-handbook-appendix.

Final recommendation: For a stakeholder submission, prioritize clear business scope, testable functional requirements, measurable NFRs, security, integrations, acceptance criteria, and traceability. Include architecture diagrams when they clarify the system, and keep detailed implementation design separate unless requested.